T3000 API & Protocol

Inputs · built Nov 13, 2026

Inputs

Monitor and manage analog and digital input points for sensors and measurements.

Overview

Input points represent physical sensors connected to the BACnet device, including temperature sensors, pressure transducers, flow meters, and digital status inputs.

Input Types

Analog Inputs (AI)

Continuous measurement values:

Common Sensor Types:

- **Temperature**: Thermistors, RTDs, thermocouples
- **Pressure**: 4-20mA transducers, 0-10V sensors
- **Flow**: Differential pressure, ultrasonic
- **Humidity**: Relative humidity sensors
- **CO2**: Carbon dioxide sensors (PPM)
- **Voltage/Current**: Direct electrical measurements

Properties:

- **Present Value**: Current reading
- **Units**: Engineering units (°F, °C, PSI, CFM, etc.)
- **Range**: Min/max values (e.g., 0-100%)
- **Filter**: Averaging time constant
- **Calibration**: Offset and gain

Binary Inputs (BI)

On/off status readings:

Common Uses:

- **Status Switches**: Proof of operation
- **Door Contacts**: Open/closed sensors
- **Limit Switches**: End-of-travel indicators
- **Alarm Contacts**: Safety signals
- **Occupancy Sensors**: Motion detection

States:

- **Active** (1, On, Closed)
- **Inactive** (0, Off, Open)

Inputs Page Interface

Table View

#	Label	Description	Value	Units	Status	Auto/Manual	Last Update
1	Zone_Temp	Zone Temperature	72.5	°F	✅ Valid	Auto	2s ago
2	Supply_Press	Supply Pressure	45.2	PSI	✅ Valid	Auto	2s ago
3	Return_Temp	Return Air Temp	68.0	°F	✅ Valid	Auto	2s ago
4	CO2_Level	CO2 Concentration	650	PPM	✅ Valid	Auto	2s ago
5	Occupancy	Room Occupied	Active	-	✅ Valid	Auto	2s ago

Features

Real-Time Updates:

- Auto-refresh every 5 seconds (configurable)
- Manual refresh with 🔄 button
- Loading indicator during refresh

Batch Operations:

- Select multiple points
- Batch save changes
- Export selected points

Filtering & Search:

- Quick search by name/description
- Filter by type (AI/BI)

- Filter by status (valid/fault/override)
- Sort by any column

Input Configuration

Basic Settings

Click on an input row to configure:

Identification:

- **Label**: User-friendly name (e.g., "Zone Temperature")
- **Description**: Detailed description
- **Panel**: Associated control panel
- **Floor**: Physical location

Hardware:

- **Sensor Type**: Select sensor model
- **Range**: Measurement range (0-10V, 4-20mA, etc.)
- **Units**: Engineering units
- **Resolution**: Decimal places

Advanced Settings

Calibration:

```
TEXT
```

```
Raw Value:     5.25V
Offset:        -0.05V
Gain:          1.02
Calibrated:    5.26V
Scaled Value:  72.5°F
```

Filtering:

- **Filter Time**: Smoothing period (seconds)
- **Average Samples**: Number of readings to average
- **Deadband**: Minimum change to report

Limits:

- **High Limit**: Maximum acceptable value

- **Low Limit**: Minimum acceptable value
- **High Alarm**: Alarm threshold (upper)
- **Low Alarm**: Alarm threshold (lower)

Data Quality

Status Indicators

- ✅ **Valid**: Normal operation
- ⚠️ **Uncertain**: Questionable data quality
- ❌ **Fault**: Sensor error or out of range
- 🔧 **Override**: Manual override active

Fault Detection

Common Faults:

- **Open Circuit**: Sensor disconnected
- **Short Circuit**: Wiring fault
- **Out of Range**: Value exceeds limits
- **Communication Loss**: No recent updates

Fault Response:

- Display fault status
- Generate alarm
- Use default value (if configured)
- Notify operators

Input Trends

Real-Time Monitoring

View input trends in real-time:

1. Navigate to **Trend Logs** page
2. Select input points to chart
3. View live updating graphs
4. Zoom/pan to analyze patterns

Historical Data

Access stored input data:

- **Time Range**: Select date/time period
- **Export**: Download CSV/Excel
- **Statistics**: Min, max, avg, std dev
- **Comparison**: Plot multiple inputs

Alarms

Alarm Configuration

Set alarm thresholds for inputs:

High Alarm:

```
TEXT

Setpoint:     80°F
Deadband:     2°F
Priority:     Medium
Delay:        60 seconds
```

Low Alarm:

```
TEXT

Setpoint:     65°F
Deadband:     2°F
Priority:     Low
Delay:        60 seconds
```

Alarm Actions

When alarm triggers:

- Visual indication in inputs table
- Entry in alarm log
- Email/SMS notification (if configured)
- Audible alert (if enabled)

Best Practices

Naming Conventions

Use consistent naming:

```
Format: [Location]_[Function]_[Measurement]

Examples:
- AHU1_Zone_Temp
- Boiler_Supply_Press
- Floor3_CO2_Level
```

Calibration

- Calibrate sensors annually
- Document calibration date
- Compare to known reference
- Record offset/gain values

Maintenance

1. **Regular Checks**
   - Verify readings match physical conditions
   - Check for drift over time
   - Test alarm functions
2. **Documentation**
   - Maintain sensor specifications
   - Record installation dates
   - Track maintenance history
3. **Spare Parts**
   - Stock common sensor types
   - Keep calibration tools available
   - Document sensor part numbers

Troubleshooting

Reading Errors

Symptom: Incorrect or unstable readings

Solutions:

1. Check sensor wiring
2. Verify sensor type configuration
3. Review calibration settings
4. Test with multimeter
5. Replace faulty sensor

No Updates

Symptom: Stale data, no refresh

Solutions:

1. Check device connection
2. Verify auto-refresh enabled
3. Check polling interval
4. Review communication logs

Alarm Nuisance

Symptom: False alarms

Solutions:

1. Adjust alarm thresholds
2. Increase deadband
3. Add time delay
4. Improve sensor filtering

Next Steps

- Outputs - Control output points
- Variables - Calculated variables
- Trend Logs - Historical analysis
- Alarms - Alarm management

Inputs

BACnet Input Objects

Reading Input Values via API

TYPESCRIPT

```typescript
// Read single input
const response = await fetch(
  `http://localhost:9103/api/devices/389001/points/IN1`
);
const data = await response.json();

console.log(`Value: ${data.value} ${data.units}`);
console.log(`Quality: ${data.quality}`);
console.log(`Timestamp: ${new Date(data.timestamp)}`);

// Read multiple inputs in batch
const points = ['IN1', 'IN2', 'IN3', 'IN4'];
const batchResponse = await fetch(
  `http://localhost:9103/api/devices/389001/points/batch`,
  {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ points })
  }
);
```

BACnet Property Reading

TYPESCRIPT

```typescript
import { BACnetClient } from '@temco/t3000-sdk';

const client = new BACnetClient({ port: 47808 });

// Read present-value property
const value = await client.readProperty({
  deviceInstance: 389001,
  objectType: 'analog-input',
  objectInstance: 1,
  property: 'present-value'
});

// Read multiple properties
const properties = await client.readPropertyMultiple({
  deviceInstance: 389001,
  objects: [
    {
      objectId: { type: 'analog-input', instance: 1 },
      properties: [
        'present-value',
        'status-flags',
        'out-of-service',
        'units',
        'reliability'
      ]
    }
  ]
});
```

Input Signal Processing

Analog Input Scaling

TYPESCRIPT

```typescript
interface ScalingConfig {
  rawMin: number;
  rawMax: number;
  engMin: number;
  engMax: number;
  offset: number;
  gain: number;
}

function scaleAnalogInput(rawValue: number, config: ScalingConfig): number {
  // Apply offset and gain calibration
  const calibrated = (rawValue + config.offset) * config.gain;

  // Scale to engineering units
  const scaled = config.engMin +
    ((calibrated - config.rawMin) / (config.rawMax - config.rawMin)) *
    (config.engMax - config.engMin);

  return scaled;
}

// Example: 0-10V sensor scaled to 0-100°F with calibration
const config: ScalingConfig = {
  rawMin: 0,
  rawMax: 10,
  engMin: 0,
  engMax: 100,
  offset: -0.05,  // -0.05V offset
  gain: 1.02      // 2% gain correction
};

const temperature = scaleAnalogInput(5.25, config); // Returns ~52.6°F
```

Digital Filtering

TYPESCRIPT

```typescript
class ExponentialFilter {
  private filtered: number;
  private alpha: number;

  constructor(filterTime: number, sampleRate: number) {
    // Calculate alpha from filter time constant
    this.alpha = 1 - Math.exp(-sampleRate / filterTime);
    this.filtered = 0;
  }

  update(newValue: number): number {
    if (this.filtered === 0) {
      this.filtered = newValue;  // Initialize
    } else {
      this.filtered = this.alpha * newValue + (1 - this.alpha) * this.filtered;
    }
    return this.filtered;
  }
}

// Example: 10-second filter at 1Hz sample rate
const filter = new ExponentialFilter(10, 1);

// Apply to incoming sensor data
const rawValues = [72.1, 72.5, 71.8, 72.3, 72.0];
const filtered = rawValues.map(v => filter.update(v));
console.log(filtered); // Smoothed values
```

Moving Average Filter

TYPESCRIPT

```typescript
class MovingAverageFilter {
  private samples: number[] = [];
  private size: number;

  constructor(windowSize: number) {
    this.size = windowSize;
  }

  update(newValue: number): number {
    this.samples.push(newValue);

    if (this.samples.length > this.size) {
      this.samples.shift();
    }

    return this.samples.reduce((a, b) => a + b, 0) / this.samples.length;
  }
}
```

Fault Detection Algorithms

Range Checking

TYPESCRIPT

```typescript
interface InputLimits {
  sensorMin: number;
  sensorMax: number;
  physicalMin: number;
  physicalMax: number;
}

function detectRangeFault(value: number, limits: InputLimits): string | null {
  if (value < limits.sensorMin || value > limits.sensorMax) {
    return 'SENSOR_FAULT';  // Hardware fault
  }

  if (value < limits.physicalMin || value > limits.physicalMax) {
    return 'OUT_OF_RANGE';  // Physically impossible value
  }

  return null;  // No fault
}

// Example: Temperature sensor
const limits: InputLimits = {
  sensorMin: -40,    // Sensor spec
  sensorMax: 140,
  physicalMin: 50,   // Expected building range
  physicalMax: 90
};

const fault = detectRangeFault(150, limits); // Returns 'SENSOR_FAULT'
```

Rate of Change Detection

TYPESCRIPT

```typescript
class RateOfChangeDetector {
  private lastValue: number | null = null;
  private lastTime: number | null = null;

  check(value: number, maxRate: number): boolean {
    const now = Date.now();

    if (this.lastValue !== null && this.lastTime !== null) {
      const deltaValue = Math.abs(value - this.lastValue);
      const deltaTime = (now - this.lastTime) / 1000; // seconds
      const rate = deltaValue / deltaTime;

      if (rate > maxRate) {
        return true;  // Excessive rate of change
      }
    }

    this.lastValue = value;
    this.lastTime = now;
    return false;
  }
}

// Example: Detect temperature changing faster than 5°F/minute
const rateDetector = new RateOfChangeDetector();
const excessive = rateDetector.check(78, 5/60); // 5°F per minute = 0.083°F/s
```

Data Logging and Storage

High-Performance Trend Logging

TYPESCRIPT

```typescript
import Database from 'better-sqlite3';

class TrendLogger {
  private db: Database.Database;
  private buffer: Map<string, Array<{value: number, timestamp: number}>>;
  private batchSize = 100;

  constructor(dbPath: string) {
    this.db = new Database(dbPath);
    this.buffer = new Map();

    // Create optimized table
    this.db.exec(`
      CREATE TABLE IF NOT EXISTS input_trends (
        device_id INTEGER NOT NULL,
        point_id TEXT NOT NULL,
        value REAL NOT NULL,
        timestamp INTEGER NOT NULL
      );

      CREATE INDEX IF NOT EXISTS idx_input_trends
        ON input_trends(device_id, point_id, timestamp DESC);
    `);
  }

  log(deviceId: number, pointId: string, value: number) {
    const key = `${deviceId}:${pointId}`;

    if (!this.buffer.has(key)) {
      this.buffer.set(key, []);
    }

    this.buffer.get(key)!.push({
      value,
      timestamp: Date.now()
    });
```

```
    // Flush when buffer is full
    if (this.buffer.get(key)!.length >= this.batchSize) {
      this.flush(deviceId, pointId);
    }
  }

  private flush(deviceId: number, pointId: string) {
    const key = `${deviceId}:${pointId}`;
    const data = this.buffer.get(key);

    if (!data || data.length === 0) return;

    const insert = this.db.prepare(`
      INSERT INTO input_trends (device_id, point_id, value, timestamp)
      VALUES (?, ?, ?, ?)
    `);

    const transaction = this.db.transaction((entries) => {
      for (const entry of entries) {
        insert.run(deviceId, pointId, entry.value, entry.timestamp);
      }
    });

    transaction(data);
    this.buffer.set(key, []);
  }
}
```

WebSocket Real-Time Streaming

TYPESCRIPT

```typescript
// Subscribe to input updates
const ws = new WebSocket('ws://localhost:9103/ws');

ws.onopen = () => {
  ws.send(JSON.stringify({
    type: 'subscribe',
    deviceId: 389001,
    points: ['IN1', 'IN2', 'IN3'],
    changeOfValue: 0.5,  // Only send if value changes by 0.5 or more
    interval: 5000        // Maximum update interval (ms)
  }));
};

ws.onmessage = (event) => {
  const data = JSON.parse(event.data);

  if (data.type === 'point-update') {
    console.log(`${data.point}: ${data.value} (COV: ${data.cov})`);
    updateChart(data.point, data.value, data.timestamp);
  }
};
```

Next Steps

- REST API Reference
- WebSocket API
- Modbus Protocol